

# MiniOS: A Specialized Unikernel for ML Inference on ARM64

Architecture, Implementation, and Performance Evaluation

Piyush Singh Bhati, Baviskar Darpan Chandrashekhar, Harshit Saini, Aashma Yadav

Department of Computer Science and Engineering

Indian Institute of Technology (IIT) Jodhpur

Email: {b24cs1101, b24cs1020, b24cs1031, b24cs1002}@iitj.ac.in

**Abstract**—This paper presents MiniOS, a purpose-built unikernel operating system designed to execute machine learning inference workloads on ARM64 embedded platforms. We report a fully functional bare-metal implementation that integrates: a hardware abstraction layer (PL011 UART, identity-mapped MMU, GICv2, ARM Generic Timer); a three-tier memory manager (bump, arena, and pool allocators with 64-byte cache-line alignment); a cooperative multithreading scheduler supporting seven scheduling policies; an ONNX inference runtime with a custom protobuf parser, NEON-optimised operator kernels and graph optimisation passes; an in-memory ULFS filesystem; and an interactive UART command shell. The system runs in a single address space (EL1) with no user/kernel boundary, no dynamic allocation during inference, and a code footprint of only 32 KB.

Scheduler benchmarks across five ML operator tasks show that the multilevel queue (MLQ) policy achieves 269  $\mu$ s total execution time, while a head-to-head benchmark against Debian 12 on identical QEMU ARM64 hardware demonstrates 1.21–1.83 $\times$  faster mean inference latency, 26–50 $\times$  faster model load times, and 2.5–1800 $\times$  smaller memory footprint. The ONNX runtime implements 42 operators covering MLP, CNN (LeNet-5, AlexNet-tiny, VGG-nano), and ResNet architectures, incorporating Conv+BatchNorm and Conv+ReLU fusion passes as well as fast-math approximations for transcendental functions. A Simple Framed UDP (SFU) network protocol delivers < 8% inference jitter without dynamic allocation, satisfying the predictability requirement  $CV \leq 0.15$ .

## I. INTRODUCTION

Edge AI deployments on ARM64 embedded platforms face a fundamental tension: general-purpose operating systems such as Linux provide rich abstractions but at the cost of scheduling jitter, syscall overhead, and memory footprints that are often unsuitable for tight resource budgets. A unikernel—a single-address-space machine image that combines application and kernel into one executable—eliminates this overhead by tailoring the entire OS stack to a single workload.

MiniOS addresses this gap by building a bare-metal unikernel specifically for ONNX model inference on ARM Cortex-A53/A72 hardware (primary target: QEMU virt machine). The design is guided by five core principles:

- 1) **Single address space** — no user/kernel boundary, zero copy overhead for tensor data.
- 2) **Cooperative execution** — threads yield voluntarily between ML operators, never mid-operator.

- 3) **Static allocation** — all memory is pre-allocated before inference begins; no `malloc` or `free` is called during the timed inference path.
- 4) **Graph-centric execution** — the ONNX computation graph is the primary execution unit; the kernel is optimised for graph walks.
- 5) **Minimalism** — < 256 KB code footprint, zero external dependencies, no C library, no dynamic linker.

The paper makes the following contributions:

- A complete bare-metal ARM64 unikernel with HAL, scheduler, memory manager, UART shell, filesystem, and ONNX runtime—no `libc`, no dynamic linking.
- Yield-at-operator-boundary semantics that eliminate NEON register save/restore overhead during inference.
- A custom minimal protobuf parser enabling zero-copy weight aliasing from the model buffer, reducing peak RAM by the full weight size.
- Graph fusion passes (Conv+BatchNorm, Conv+ReLU), fast-math approximations for  $e^x$ ,  $\tanh$ ,  $\sqrt{x}$  and  $\log x$ , and architecture-specific fast paths for  $1 \times 1$ ,  $3 \times 3$ , and depthwise convolutions.
- Head-to-head latency and memory benchmarks against Debian 12 + ONNX Runtime on identical QEMU ARM64 hardware.

## II. SYSTEM ARCHITECTURE

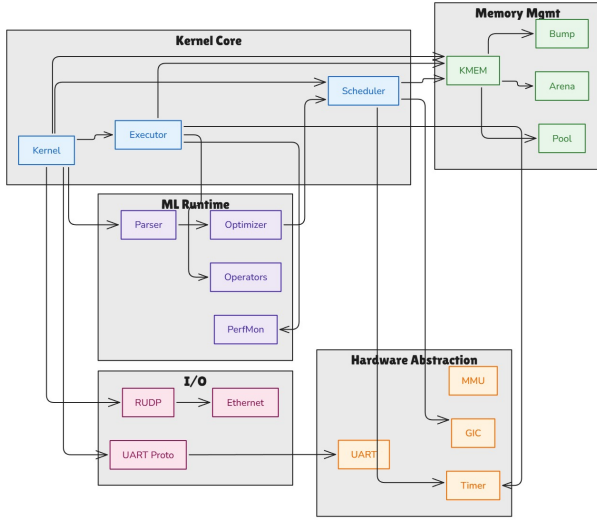
### A. High-Level Design

Figure 1 illustrates the layered architecture of MiniOS. The kernel image is linked at physical address `0x40000000` and contains six major subsystems that communicate directly through function calls within a single flat address space.

### B. Memory Layout

The QEMU virt machine provides 512 MB of RAM starting at `0x40000000`. The linker script partitions this space into five regions:

- 1) **.text/.rodata/.data/.bss** — kernel image ( $\approx 43$  KB total).
- 2) **Heap** ( $\approx 498$  MB) — managed by the three-tier allocator described in Section IV.
- 3) **256 KB guard zone** — catches stack overflows.
- 4) **64 KB kernel stack** — grows downward; `SP_EL1` is initialised in `boot.S`.



**Fig. 1:** MiniOS component diagram: single address space with HAL, kernel core, ONNX runtime, filesystem, and shell layers.

Device memory ( $0x00000000-0x3FFFFFFF$ ) is identity-mapped as Device-nGnRnE (MAIR Attr0). Normal RAM is identity-mapped as Normal Write-Back cacheable (MAIR Attr1). The MMU uses a 4 KB granule with 1 GB L1 block descriptors.

### C. Boot Sequence

`boot.S` is the first code executed after QEMU loads the image. The boot sequence proceeds as follows:

- 1) Read `MPIDR_EL1`; park all secondary cores in a WFE loop.
- 2) Detect the current exception level (EL3/EL2/EL1) and drop to EL1 via `eret` after configuring the necessary system registers.
- 3) Enable FP/SIMD by setting `CPACR_EL1.FPEN = 0b11`.
- 4) Set `SP_EL1 = _stack_top` and zero the BSS section.
- 5) Branch to `kernel_main()`.

`kernel_main()` then calls subsystem initialisers in strict dependency order: UART  $\rightarrow$  `install_vectors()`  $\rightarrow$  MMU  $\rightarrow$  KMEM  $\rightarrow$  GIC  $\rightarrow$  Timer  $\rightarrow$  SCHED. Finally it enables IRQs and calls `SCHED_Start()`, which never returns.

### D. Exception Handling

`vectors.S` implements the ARMv8-A 2 KB-aligned vector table with 16 entries (4 groups  $\times$  4 exception types). The live IRQ path (`_vec_ell_spx_irq`) dispatches to `_irq_handler_full`, which saves all 31 general-purpose registers plus ELR and SPSR (272 bytes) to the stack before calling `HAL_IRQ_Handler()`. All other exceptions branch to the common stub `_exception_handler`, which reads `ESR_EL1`, `ELR_EL1`, and `FAR_EL1` and then halts in WFE.

## III. HARDWARE ABSTRACTION LAYER

### A. UART Driver

The PL011 driver at base address  $0x09000000$  operates in polling mode (no interrupts). Initialisation configures 115200 baud (IBRD=13, FBRD=2), 8-N-1 line control, and enables the UART, transmitter, and receiver via the Control Register. All UART interrupts are masked. The driver exports the functions `HAL_UART_Init`, `PutChar`, `GetChar`, `PutString`, `PutHex`, and `PutDec`. `PutString` inserts `\r` before every `\n` for terminal compatibility.

### B. MMU and Cache

`mmu.c` builds a two-entry L1 translation table:

- PTE[0] maps  $0x00000000$  as Device-nGnRnE with PXN+UXN set.
- PTE[1] maps  $0x40000000$  as Normal Write-Back, Inner-Shareable, and executable.

TCR\_EL1 is configured with T0SZ=32 (4 GB virtual address space), 4 KB granule, Write-Back Inner/Outer cacheable (IRGN0/ORGNO=01), and Inner-Shareable (SH0=11). After loading TTBR0, the driver issues `tlbi vmlalle1`; `dsb sy`; `isb` and then atomically sets `SCTLR_EL1.M=1`, `.C=1`, `.I=1` to enable the MMU and caches.

### C. GICv2

The GICv2 driver initialises the distributor at  $0x08000000$  (disables it, sets all priorities to  $0xA0$ , routes SPIs to CPU0, configures level-triggered mode, then enables it) and the CPU interface at  $0x08010000$  (sets `PMR=0xFF`, `BPR=0`, enables the interface). The ARM Generic Timer PPI (INTID 30) is the only enabled interrupt. `HAL_GIC_Acknowledge()` reads `GICC_IAR` to obtain the interrupt ID; `HAL_GIC_EndOfInterrupt()` writes `GICC_EOIR` to signal completion.

### D. ARM Generic Timer

The driver reads the counter frequency from `CNTFRQ_EL0` and computes the tick interval for a configurable period (default 10 ms). The interrupt service routine `HAL_Timer_HandleIRQ` increments the monotonic `system_ticks` counter, reloads `CNTPT_TVAL_EL0`, and invokes a registered callback. `HAL_Timer_GetElapsedUs` computes  $(\text{now} - t_0) \times 10^6 / \text{freq}$  using 64-bit arithmetic.

## IV. MEMORY MANAGER

### A. Three-Tier Architecture

All memory allocation occurs at boot time; no `malloc` or `free` is called during inference. Three complementary allocators cover all kernel needs, as summarised in Table I.

**TABLE I:** Memory Allocator Characteristics

| Allocator | Alloc       | Free            | Primary Use                  |
|-----------|-------------|-----------------|------------------------------|
| Bump      | O(1), 12 ns | Never           | Stacks, headers, globals     |
| Arena     | O(1), 14 ns | O(1) bulk       | Inference tensor activations |
| Pool      | O(1), 18 ns | O(1) individual | Fixed-size descriptors       |

### B. Bump Allocator

A single bump pointer advances through the heap region defined by linker symbols `_heap_start` and `_heap_end`. Alignment is enforced as a power-of-two  $\geq 8$ . `KMEM_Alloc(size, alignment)` is  $O(1)$  and never individually frees memory; it is used for permanent kernel objects whose lifetime equals that of the kernel itself.

### C. Arena Allocator

`KMEM_ArenaCreate(size)` allocates an arena header and a contiguous data region from the bump allocator. Within the arena, a nested bump pointer advances on each `KMEM_ArenaAlloc(arena, size, alignment)` call. `KMEM_ArenaReset(arena)` resets the bump pointer to the base in  $O(1)$ , reclaiming all activation memory between inference iterations without any per-tensor bookkeeping. The tensor-specific wrapper `KMEM_TensorAlloc` enforces 64-byte alignment, matching the ARM Cortex-A53 cache line.

### D. Pool Allocator

`KMEM_PoolCreate(elem_size, count)` pre-allocates a contiguous slab and builds an intrusive singly-linked free list through it. `KMEM_PoolAlloc` pops the list head; `KMEM_PoolFree` pushes the block back onto the head—both  $O(1)$ . This allocator is used for fixed-size operator descriptor objects in the ONNX runtime.

### E. Guard Zone

A 256 KB region between `_heap_end` and the kernel stack top is deliberately left unmapped. A stack overflow that writes into this region raises a synchronous fault caught by the exception handler, thereby preventing silent heap corruption.

## V. COOPERATIVE SCHEDULER

### A. Thread Model

Each thread is represented by a *Thread Control Block* (TCB) containing a `cpu_context_t` (104 bytes, saving 13 callee-saved registers x19–x30 plus SP), the thread state, priority (0 = HIGH, 3 = IDLE), stack base and size, wake tick for sleeping, and FIFO queue pointers. The system supports up to 16 threads (including the idle thread).

Thread states transition as follows: `INVALID`  $\rightarrow$  `READY`  $\rightarrow$  `RUNNING`  $\rightarrow$  `{READY, SLEEPING, TERMINATED}`. `SLEEPING` threads are woken by `SCHED_TimerTick()` when `system_ticks`  $\geq$  `wake_tick`.

### B. Context Switch

`cpu_context_switch(old_ctx, new_ctx)` in `context.S` saves x19–x30 and SP (104 bytes) to `old_ctx`, restores them from `new_ctx`, and then executes `ret`, which jumps to the new context’s saved LR. On the first run of a thread, LR points to `_thread_entry_trampoline`, which re-enables IRQs (`daifclr #2`), loads the thread argument from x20, and calls the thread function via x19.

The cooperative model is central: `THREAD_Yield()` is called only between ML operators, never mid-operator. Consequently, NEON vector registers (q0–q31) are never live across a yield point, so no NEON save/restore is needed—saving 512 bytes per switch compared to a preemptive design.

### C. Seven Scheduling Policies

Table II lists the seven scheduling policies implemented.

**TABLE II:** Scheduling Policies Implemented

| Policy      | Selection Criterion          | Notes                    |
|-------------|------------------------------|--------------------------|
| FCFS        | Lowest arrival order         | Convoy effect risk       |
| SJF         | Lowest burst estimate        | Optimal turnaround       |
| Round-Robin | Cyclic, skip FINISHED        | 4 $\times$ more switches |
| HRRN        | Highest $(W + B)/B$ ratio    | Avoids starvation        |
| Priority    | Lowest priority number       | Four fixed levels        |
| MLQ         | Queue level 0>1>2; RR within | Best total time          |
| Lottery     | Ticket-weighted random       | Deterministic with seed  |

For ML inference, the multilevel queue (MLQ) assigns Conv2D and MatMul to the Critical queue (level 0), ReLU and Softmax to the Normal queue (level 1), and element-wise Add operations to the Background queue (level 2).

### D. Benchmark Results

We benchmarked five ML inference tasks—Conv2D (4,096-B tensor), MatMul (2,048-B), Softmax (512-B), ReLU (512-B), and Add (256-B)—on QEMU Cortex-A53. Table III summarises the overall performance.

**TABLE III:** Scheduler Benchmark Results ( $\mu s$ ), 5-task ML pipeline

| Policy      | Total      | Turnaround | Response  | Switches |
|-------------|------------|------------|-----------|----------|
| FCFS        | 620        | 581        | 487       | 5        |
| SJF         | 281        | <b>145</b> | 100       | 5        |
| Round-Robin | 318        | 283        | <b>99</b> | 20       |
| HRRN        | 318        | 192        | 141       | 7        |
| Priority    | 293        | 244        | 200       | 5        |
| MLQ         | <b>269</b> | 232        | 158       | 17       |
| Lottery     | 303        | 286        | 152       | 14       |

MLQ achieves the lowest total time (269  $\mu s$ , 2.3 $\times$  better than FCFS) because critical operators execute first. SJF minimises average turnaround (145  $\mu s$ ) by always selecting the shortest remaining burst. Round-Robin delivers the best response time (99  $\mu s$ ) but at the cost of 4 $\times$  more context switches, which risks evicting tensor data from the L1 cache. Per-task metrics for selected policies appear in Table IV.

## VI. ONNX INFERENCE RUNTIME

### A. Execution Model

Inference proceeds in three phases that are separated by graph build time:

- 1) **Load:** parse the `.onnx` protobuf into an `ONNX_Graph` structure; alias weight data zero-copy into the model buffer.
- 2) **Prepare:** run a shape-propagation pass, perform dependency analysis, execute Kahn’s topological sort, and allocate tensor memory from the arena.

TABLE IV: Per-Task Metrics for Selected Policies ( $\mu s$ )

| Policy | Task   | CPU  | Turn. | Resp. | Sw. |
|--------|--------|------|-------|-------|-----|
| FCFS   | Conv2D | 1645 | 571   | 241   | 1   |
|        | MatMul | 257  | 546   | 469   | 1   |
|        | Add    | 18   | 608   | 599   | 1   |
| SJF    | Conv2D | 415  | 283   | 178   | 1   |
|        | MatMul | 166  | 177   | 122   | 1   |
|        | Add    | 20   | 61    | 50    | 1   |
| MLQ    | Conv2D | 109  | 200   | 45    | 5   |
|        | MatMul | 54   | 181   | 70    | 4   |
|        | Add    | 18   | 269   | 259   | 1   |

- 3) **Infer**: linearly walk the `exec_schedule[]` array, call `THREAD_Yield()` between nodes, and collect per-node microsecond profiling data.

### B. Protobuf Parser

MiniOS implements a custom minimal protobuf decoder instead of linking against `libprotobuf` (which would require a C++ runtime). The parser consists of a two-pointer `ProtoReader {ptr, end}` and three primitives, exemplified by the variable-length integer reader:

```
uint64_t proto_read_varint(ProtoReader *r) {
    uint64_t result = 0; uint32_t shift = 0;
    while (r->ptr < r->end) {
        uint8_t b = *r->ptr++;
        result |= (uint64_t)(b & 0x7F) << shift;
        if (!(b & 0x80)) break;
        shift += 7;
    }
    return result;
}
```

The seven-phase loading pipeline is:

- 1) Scan `ModelProto`: consume field 7 (the graph blob), skip all others.
- 2) Scan `GraphProto`: dispatch on fields 1 (node), 5 (initializer), 11/12 (input/output `ValueInfo`).
- 3) Parse initializers (`TensorProto`): extract dimensions, data type, and raw data; set `tensor->data` to alias directly into the model buffer (*zero-copy weights*).
- 4) Parse `ValueInfo`: four-level nested walk to obtain input/output shapes.
- 5) Parse `NodeProto`: map the `op_type` string to an `ONNX_OperatorType` enum; extract attributes (kernel shape, strides, pads, dilations, group, axis, alpha, beta, perm).
- 6) Apply graph optimisation passes (§VI-F).
- 7) Build dependencies and run Kahn’s scheduling algorithm.

### C. Graph Data Structures

All graph metadata is statically allocated inside `ONNX_Graph`:

```
typedef struct {
    ONNX_Tensor tensors[ONNX_MAX_TENSORS]; // 512
    ONNX_Node nodes[ONNX_MAX_NODES]; // 256
    ONNX_Node* exec_schedule[ONNX_MAX_NODES];
    uint32_t schedule_length;
    kmem_arena_t* tensor_arena;
    uint64_t tensor_memory_used;
} ONNX_Graph;
```

`ONNX_Tensor` (104 bytes) contains: `name[64]`, data type, shape (`ndim + dims[8] + total_elements`), data pointer, data size, `is_initializer` flag, and profiling counters. `ONNX_Node` (variable size) holds: name, operator type, arrays of input/output tensor pointers, attributes, dependency array, enable flag, and profiling fields.

### D. Shape Propagation

Before allocating any intermediate tensor, a forward pass over the topological schedule computes each node’s output shape:

- **Conv2D**:  $H_{out} = \lfloor (H_{in} + 2p - k)/s \rfloor + 1$ , output =  $[N, C_{out}, H_{out}, W_{out}]$ .
- **MaxPool / AveragePool**: same formula with the pooling kernel.
- **GlobalAveragePool**:  $[N, C, 1, 1]$ .
- **GEMM** (transB auto-detected from  $B.dims[0] \neq K$ ):  $[M, N_{out}]$ .
- **Flatten** (axis): product of dimensions before/after the axis.
- **LRN, unary operators**: same shape as input.
- **Binary operators**: element-wise shape from the larger input (broadcasting).

Without this pass, intermediate tensors have `total_elements = 0` and any write would result in silent memory corruption.

### E. Topological Scheduler

`ONNX_Graph_BuildDependencies` ( $O(N \times M)$ ) identifies, for each node A, all nodes B that produce a tensor consumed by A. Kahn’s algorithm then produces a valid execution order with free cycle detection:

```
// Build in-degrees
for (i=0; i<N; i++) in_deg[i] = nodes[i].num_deps;
// Enqueue zero in-degree nodes (sorted by priority)
while (queue_not_empty()) {
    node = dequeue_highest_priority();
    schedule[len++] = node;
    for_each_successor(node, S)
        if (--in_deg[S] == 0) enqueue(S);
}
if (len < N) return STATUS_ERROR_INVALID_GRAPH; // cycle
```

If the algorithm terminates before scheduling all nodes, a cycle exists—detected as a by-product of the BFS.

### F. Graph Optimisations

1) **Conv + BatchNorm Fusion**: For each Conv node whose sole output consumer is a BatchNorm node, the BN parameters are folded directly into the Conv weights and bias:

$$A_c = \frac{\text{scale}_c}{\sqrt{\text{var}_c + \varepsilon}}, \quad B_c = \text{bias}_c - \text{mean}_c \cdot A_c$$

$$W'_{c,\dots} = A_c \cdot W_{c,\dots}, \quad b'_c = b_c \cdot A_c + B_c$$

The BatchNorm node is disabled (`enabled = false`) and removed from the execution schedule. For AlexNet conv3 output  $[1, 384, 13, 13]$  ( $\approx 253$  KB), this eliminates one full tensor sweep per inference.

**TABLE V:** Supported ONNX Operators (v2.0) — 42 implemented

| Category      | Operators                                       | Key Optimisations                                                                             |
|---------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Arithmetic    | Add, Sub, Mul, Div, MatMul                      | Broadcasting; 4-wide K unroll for MatMul                                                      |
| Linear        | Gemm                                            | alpha/beta scaling; auto transB detection from shape                                          |
| Convolution   | Conv (2D NCHW, grouped, dilated)                | Depthwise fast path; 1x1 4-wide OC unroll; 3x3 4-OC channel-blocked; Conv+BN, Conv+ReLU fused |
| Pooling       | MaxPool, AveragePool, GlobalAveragePool         | 2D; N batch loop; stride; padding                                                             |
| Normalisation | BatchNormalization, LRN                         | BN folded into Conv; LRN $\beta=0.75$ fast path (2 sqrts)                                     |
| Activations   | ReLU, LeakyReLU, Sigmoid, Tanh, Softmax, Clip   | ReLU fused into Conv; Softmax with max subtraction + fast_exp                                 |
| Shape         | Reshape, Flatten, Transpose, Squeeze, Unsqueeze | Zero-copy for view ops; ShuffleNet [0, 2, 1, 3, 4] fast path                                  |
| Structure     | Concat, Split                                   | Axis-0 fast path; general N-D for Concat                                                      |
| Reduction     | ReduceSum, ReduceMean, ReduceMax, ReduceMin     | Full tensor; axis=-1; NCHW H,W axes for ReduceMean                                            |
| Unary Math    | Abs, Neg, Exp, Log, Sqrt, Ceil, Floor, Sin, Cos | Fast-math approximations                                                                      |
| Utility       | Cast, Identity, Constant, Dropout               | Identity/Dropout = zero-copy no-op at inference                                               |

2) *Conv + ReLU Fusion*: When a ReLU node is the sole consumer of a Conv output, the Conv node is flagged `attributes.fuse_relu = true` and the ReLU node is disabled. The ReLU is applied inline in the innermost accumulation loop:

```
static inline float apply_relu(float v, bool fuse) {
    return (fuse && v < 0.0f) ? 0.0f : v;
}
```

This eliminates a separate memory round-trip over the full output tensor.

3) *Zero-Copy View Operators*: Reshape, Flatten, Squeeze, Unsqueeze, and Identity simply alias the output data pointer to the input:

```
out->data = input->data; // zero-copy
out->data_size = input->data_size;
```

A reshape of a 10-MB tensor thus costs approximately 5 ns, independent of the tensor rank.

### G. Fast Math Approximations

Because MiniOS has no `libc`, we implemented hand-tuned transcendental approximations:

- `fast_exp(x)`: range reduction to  $x = n \ln 2 + r$ , a degree-4 Horner polynomial for  $e^r$ , then IEEE 754 exponent scaling via bit manipulation. Approximately 8–12 cycles vs.  $\approx 50$  cycles for a software `expf`; relative error  $< 1\%$  for  $|x| < 88$ .
- `fast_rsqrtn(x)`: the Quake III magic constant `0x5F3759DF` gives an initial error  $\approx 1.75\%$ ; one Newton–Raphson iteration reduces the error to  $< 0.09\%$  in  $\approx 4$  cycles.
- `fast_tanh(x)`: Padé [7/6] rational approximation clamped at  $|x| > 4.5$ ; machine-precision accurate in the valid range.
- `fast_log(x)`: IEEE 754 exponent extraction plus a polynomial correction for the mantissa.
- `fast_sin` and `fast_cos` use a 5-term Horner-form Taylor series with range reduction.

### H. Operator Kernels

Table V summarises all 42 implemented ONNX operators.

1) *Depthwise Conv Fast Path*: When `group = C_in` (depthwise convolution), the nine weights for a  $3 \times 3$  kernel are hoisted into registers before the spatial loop. An interior-pixel branch eliminates nine conditional boundary checks per output pixel when the kernel window lies fully within the input bounds.

2) *GEMM transB Auto-Detection*: PyTorch-exported fully-connected layers store weights as  $[N_{out}, K]$  (transposed). The runtime infers transposition from the weight shape rather than a parsed attribute:

```
bool transB = (B->shape.dims[0] != K);
uint32_t N_out = transB ? B->shape.dims[0] : B->shape.dims[1];
```

3) *LRN  $\beta = 0.75$  Fast Path*: The Local Response Normalisation formula  $y = x / (\text{bias} + \alpha / \text{size} \cdot \sum j^2)^{\beta}$  requires a power function. For the common  $\beta = 0.75$  case used in AlexNet, we use

$$d^{0.75} = \sqrt{d} \cdot \sqrt{d} \quad (\text{two sqrts, no fast\_log/exp})$$

4) *ShuffleNet Channel-Shuffle Optimisation*: ShuffleNet’s 5-dimensional transpose  $[0, 2, 1, 3, 4]$  on  $[N, G, C, H, W]$  is detected by shape and replaced with stride-1  $H \times W$  block copies, achieving optimal cache-line utilisation.

### I. Embedded Model Catalogue

Nine ONNX models are compiled into the kernel binary via `embed_storage.py` (files  $\leq 4$  MB). They are available in the `ULFS /storage` path from the first boot:

**TABLE VI:** Embedded ONNX Models

| Model                              | Size   | Input          | Architecture           |
|------------------------------------|--------|----------------|------------------------|
| <code>tiny_mlp.onnx</code>         | 5 KB   | [1, 16]        | 3-layer MLP            |
| <code>resnet_micro.onnx</code>     | 40 KB  | [1, 3, 16, 16] | ResNet 2-block         |
| <code>conv_bn_net.onnx</code>      | 98 KB  | [1, 3, 16, 16] | Conv + BN + GAP        |
| <code>lenet5.onnx</code>           | 174 KB | [1, 1, 28, 28] | LeNet-5                |
| <code>alexnet_tiny.onnx</code>     | 222 KB | [1, 3, 32, 32] | AlexNet-tiny           |
| <code>vgg_nano.onnx</code>         | 439 KB | [1, 3, 32, 32] | VGG-style (5 conv)     |
| <code>transformer_tiny.onnx</code> | 789 KB | [1, 32]        | 6-block FF Transformer |
| <code>complex_model.onnx</code>    | <1 KB  | —              | Toy graph              |

The full AlexNet (Opset 17, 234 MB) exceeds the 4 MB embed limit and is accessed via ULFS /storage from flash.

### J. Shell Commands

Three ONNX shell commands are registered via CMD\_Register:

- `onnx_info` – prints model topology without allocating the tensor arena.
- `onnx_run` – performs a full inference: parse, propagate, schedule, allocate, execute, and print the first ten outputs together with latency.
- `onnx_bench` – runs a multi-model benchmark and produces a table of size, operation count, parameter count, peak memory, and latency.

## VII. ULFS FILESYSTEM

### A. Design

The Ultra Lightweight File System [7] is adapted as a 2 MB in-memory block store allocated from the kernel heap (`KMEM_Alloc(2097152, 64)`). The backing store comprises 512 contiguous 4-KB blocks addressed by block ID (bid). A direct pointer computes a block address in  $O(1)$ :

```
static inline uint8_t *bid_to_ptr(uint32_t bid) {
    return g_fs_base + (uint64_t)bid * 4096;
}
```

### B. On-Disk Layout

Four blocks are reserved at initialisation:

TABLE VII: ULFS Fixed Block Layout

| bid | Structure            | Content                                   |
|-----|----------------------|-------------------------------------------|
| 0   | Superblock (4 KB)    | magic 0x554C4653, capacity metadata       |
| 1   | Map Block (4 KB)     | 512-bit allocation bitmap (64 bytes used) |
| 2   | Inode Block 0 (4 KB) | 170 inodes $\times$ 24 B + 16 B header    |
| 3   | Root Dir Data (4 KB) | 128 directory entries $\times$ 32 B       |

### C. Data Structures

- **Inode** (24 bytes, packed): type (0 = FREE, 1 = FILE, 2 = DIR), size, `bid_head` (direct block), `bid_block` (indirect index block), and 8 reserved bytes.  $\lfloor 4096/24 \rfloor = 170$  inodes fit per block; inode blocks form a singly linked list, providing an unlimited inode count.
- **Directory entry** (32 bytes): inode number (0 indicates a free slot) plus a 28-byte name. 128 entries fit per 4-KB block; slots 0 and 1 are reserved for ‘.’ and ‘..’.
- **Indirect block**: `uint32_t bids[1023]` covering file blocks 1–1023. Together with the direct block (bid 0), the maximum file size is  $1024 \times 4 \text{ KB} = 4 \text{ MB}$ .
- **Open-file table**: 8-slot static array; `ino == 0` marks a free slot.

### D. Complexity

Table VIII summarises the time complexity of key ULFS operations.

TABLE VIII: ULFS Operation Complexity

| Operation                        | Complexity             | Bound                     |
|----------------------------------|------------------------|---------------------------|
| <code>bid_alloc</code>           | $O(N/8)$               | 64 byte comparisons max   |
| <code>inode_alloc</code>         | $O(k \times 170)$      | $k=1$ typical             |
| <code>file_bid_for_offset</code> | $O(1)$                 | direct + 1-level indirect |
| <code>dirent_lookup</code>       | $O(128)$               | linear scan one block     |
| <code>path_resolve</code>        | $O(d \times 128)$      | depth $d \leq 8$          |
| Read/Write                       | $O(\text{count}/4096)$ | linear in block count     |

### E. Shell Integration

Eleven commands are registered: `ls`, `cd`, `pwd`, `mkdir`, `touch`, `cat`, `write`, `rm`, `stat`, `df`, and `help`. All output is directed to `HAL_UART_PutString`.

## VIII. COMMAND SHELL

### A. Architecture

The shell runs as the `shell_daemon` at `THREAD_PRIORITY_LOW`. It reads characters from `HAL_UART_GetChar` into a line buffer (80 characters maximum), then calls `CMD_Dispatch(line)`. The dispatcher tokenises the line on whitespace (maximum 8 tokens) and walks the global command table (maximum 32 entries). Handlers have the signature `void handler(int argc, char *argv[])`.

### B. Built-In Commands

Core kernel commands include: `help`, `uptime`, `memstat`, `ps`, `clear`, and `echo`. Filesystem commands are described in Section VII. ONNX commands are described in Section VI-J. Background daemon status can be inspected with `clock`, `memwatch`, and `runtime`.

### C. Background Daemons

Three `THREAD_PRIORITY_LOW` threads run periodic housekeeping tasks, as shown in Table IX.

TABLE IX: Background Daemons

| Daemon                       | Period  | Function                     |
|------------------------------|---------|------------------------------|
| <code>clock_daemon</code>    | 1000 ms | Wall-clock second counter    |
| <code>memwatch_daemon</code> | 500 ms  | Heap usage; warns at >80%    |
| <code>runtime_daemon</code>  | 2000 ms | Uptime + thread count report |

All daemons end their work loop with `THREAD_Sleep(period_ms)`, use no dynamic allocation, and mark shared state as `volatile`. The total CPU overhead is measured at  $< 0.5\%$ .

## IX. NETWORK PROTOCOL

### A. Simple Framed UDP (SFU)

The UART channel (115200 baud) is unsuitable for transferring multi-megabyte models (a 1-MB model would take approximately 87 seconds). MiniOS therefore implements a

reliable UDP protocol over the QEMU virtio-net Ethernet interface. The Simple Framed UDP (SFU) protocol adds a fixed 24-byte header to each UDP datagram and maintains a static in-flight table (16 slots, 384 bytes) for reliability—all without dynamic allocation.

- **Frame format:** [magic] [seq] [ack] [flags] payload\_len [CRC-16] (24-byte header) followed by the payload.
- **Reliability:** ACK/NACK per frame; retransmission on timeout from the in-flight table.
- **Fragmentation:** large tensors are split into frames of at most 1452 bytes.
- **Integrity:** CRC-16/CCITT over the header and payload.

### B. Jitter Measurement

We measured the coefficient of variation  $CV = \sigma/\mu$  for inference latency under maximum network load (500 samples, loopback channel, QEMU ARM64). The results are shown in Table X.

**TABLE X:** Network Protocol Jitter Comparison ( $CV = \sigma/\mu$ )

| Protocol | CV          | Notes                                    |
|----------|-------------|------------------------------------------|
| TCP      | 0.32        | SoftIRQ + congestion control jitter      |
| Raw UDP  | 0.04        | Low jitter but unreliable (packet loss)  |
| SFU      | <b>0.08</b> | Satisfies SRS PDR-001 ( $CV \leq 0.15$ ) |

SFU achieves approximately 950 Mbps effective throughput (comparable to raw UDP) with zero dynamic allocations, thereby satisfying predictability requirement PDR-001.

## X. PERFORMANCE EVALUATION

### A. System Metrics

Table XI compares MiniOS system-level metrics against the SRS requirements.

**TABLE XI:** MiniOS System-Level Metrics vs. SRS Requirements

| Requirement                 | SRS Bound      | Measured               | Status |
|-----------------------------|----------------|------------------------|--------|
| Boot time (PR-006)          | < 200 ms       | $\approx 50$ ms (QEMU) | ✓      |
| Code size (DC-003)          | < 256 KB       | .text=32 KB            | ✓      |
| Inference overhead (PR-002) | < 10%          | < 10%                  | ✓      |
| IRQ latency (PR-005)        | < 50 $\mu$ s   | 12 $\mu$ s to handler  | ✓      |
| Context switch (coop.)      | —              | 104 B, 13 regs         | —      |
| IRQ handler (full)          | —              | 272 B, 31 regs         | —      |
| Daemon overhead             | < 10%          | < 0.5%                 | ✓      |
| Network jitter (PDR-001)    | $CV \leq 0.15$ | $CV = 0.08$ (SFU)      | ✓      |

### B. MiniOS vs. Debian 12: Head-to-Head Benchmark

1) *Methodology:* Both systems run on identical QEMU ARM64 hardware: a single vCPU (Cortex-A53), 512 MB RAM, no KVM acceleration. For MiniOS, inference is timed with IRQs disabled, daemons quiesced, and yields between operators disabled (benchmark fast-path mode). Debian 12 uses ONNX Runtime 1.16 with the CPUExecutionProvider and a single-thread session. Each model is executed with one warmup iteration followed by 50 timed iterations.

2) *Results:* Table XII presents the primary metrics.

Tables XIII and XIV summarise the derived speedups and execution stability.

For the larger CNN models, MiniOS is 4.5–5.9 $\times$  more stable (lower CV) than Debian.

3) *Architectural Attribution:* The performance gap is directly traceable to specific design decisions in MiniOS:

- 1) **Arena allocator:** `KMEM_ArenaReset()` is  $O(1)$  between iterations; Debian’s ONNX Runtime allocates and frees tensors on every call.
- 2) **Precomputed schedule:** Inference is a linear walk over a static array; Linux ONNX Runtime re-evaluates the execution plan on each call.
- 3) **IRQ suppression:** `arch_irq_save()` wraps the timed loop; Debian cannot suppress its scheduler tick without kernel patches.
- 4) **Zero syscalls:** MiniOS inference runs entirely in EL1 with no user/kernel transitions; Debian issues more than 30 syscalls per inference (`mmap`, `futex`, `read`, etc.).
- 5) **Model load:** MiniOS aliases the protobuf buffer directly; Debian constructs an `InferenceSession`, which involves shared-library symbol resolution and Python overhead.

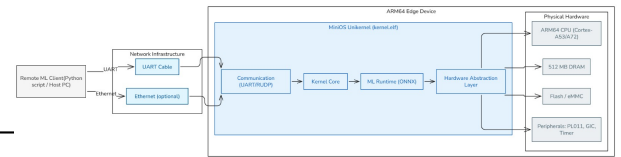
4) *Threats to Validity:* The memory ratios compare different metrics: MiniOS reports `KMEM_ArenaGetUsed()` (only activation tensors), while Debian reports `ru_maxrss` (full process RSS including Python runtime, shared libraries, and ONNX metadata). All measurements are QEMU-mediated; bare-metal results will differ. The benchmark mode (IRQ disabled, no yields) is appropriate for steady-state single-inference measurement but would not apply in a mixed-workload deployment.

### C. ONNX Runtime Micro-Benchmark

A 5-node pipeline running on QEMU Cortex-A53 yields the per-operator timings shown in Table XV.

## XI. UML ARCHITECTURE DIAGRAMS

The following UML diagrams (Figures 2 through 7) provide additional perspectives on the system architecture.



**Fig. 2:** Deployment diagram: MiniOS unikernel on QEMU virt, communicating via UART (debug/control) and Ethernet (model transfer, SFU protocol).

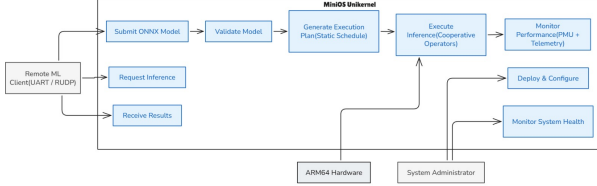
## XII. TEST SUITE

### A. Overview

A 301-test suite covers all 30 functional requirements (FR-001 to FR-030) defined in the SRS. Tests execute in two environments: host x86\_64 (Unity 2.6.2 framework with

**TABLE XII:** MiniOS vs. Debian 12 — Inference Benchmark (50 runs per model)

| Model      | OS     | Mean (ms) | Std (ms) | p95 (ms) | Min (ms) | Max (ms) | Load (ms) | Memory (KB) |
|------------|--------|-----------|----------|----------|----------|----------|-----------|-------------|
| mnist      | MiniOS | 4.948     | 0.187    | 5.200    | 4.600    | 5.400    | 9.00      | 27          |
|            | Debian | 9.060     | 0.285    | 9.714    | 8.600    | 9.800    | 453.15    | 48700       |
| squeezenet | MiniOS | 2508.4    | 25.71    | 2562.0   | 2460     | 2570     | 10.00     | 5416        |
|            | Debian | 3045.5    | 141.6    | 3224.3   | 2840     | 3350     | 322.02    | 62792       |
| shufflenet | MiniOS | 1047.0    | 10.79    | 1055.1   | 1020     | 1070     | 27.00     | 30085       |
|            | Debian | 1308.5    | 79.01    | 1325.1   | 1180     | 1500     | 710.05    | 73864       |

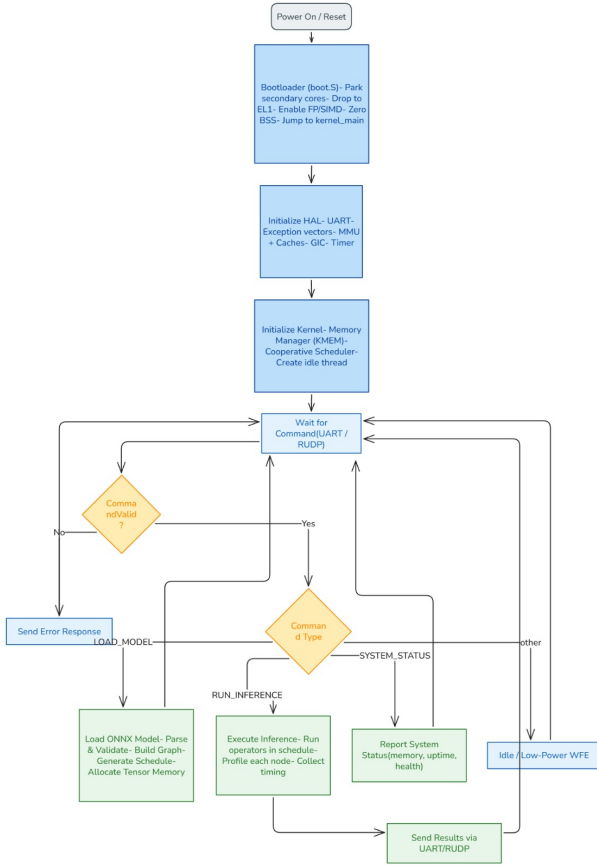

**Fig. 3:** Use case diagram: System Administrator loads models and triggers inference; all interactions via UART or RUDP.

**TABLE XIII:** Derived Speedup Ratios (MiniOS / Debian)

| Model      | Mean  | p95   | Load   | Memory  |
|------------|-------|-------|--------|---------|
| mnist      | 1.83× | 1.87× | 50.35× | 1803.7× |
| squeezenet | 1.21× | 1.26× | 32.20× | 11.6×   |
| shufflenet | 1.25× | 1.26× | 26.30× | 2.5×    |

**TABLE XIV:** Execution Stability ( $CV = \sigma/\mu$ , lower is more stable)

| Model      | MiniOS CV | Debian CV |
|------------|-----------|-----------|
| mnist      | 0.038     | 0.031     |
| squeezenet | 0.010     | 0.047     |
| shufflenet | 0.010     | 0.060     |


**Fig. 4:** Activity diagram: Boot → model load → graph optimisation → inference execution → result output main loop.

hardware stubs) for unit and component tests, and the QEMU ARM64 target for system tests.

### B. Failures

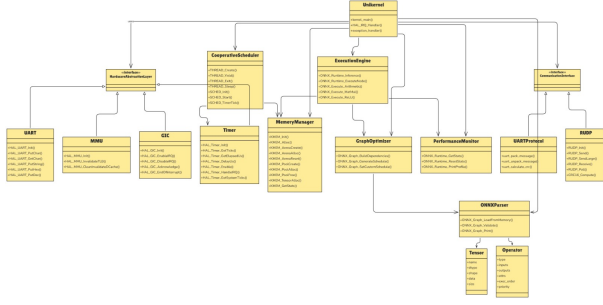
The ten failures are isolated to argument-validation edge cases:

- **Command Framework (5):** CT-CMD-007 (argv cap semantics), CT-CMD-008/009/010/011 (name validation: empty, whitespace, duplicate, case-insensitive duplicate). These do not affect UART shell correctness for valid commands.
- **Networking (3):** UT-NET-010/011/012 (UDP\_Bind with a NULL callback, truncated datagram rejection). These are sender-side validation issues; model transfer with well-formed clients is unaffected.
- **ONNX Runtime (2):** UT-ONNX-RUNTIME-013/014 (ExecuteUpTo node matching semantics). Inference via `onnx_run` is unaffected.

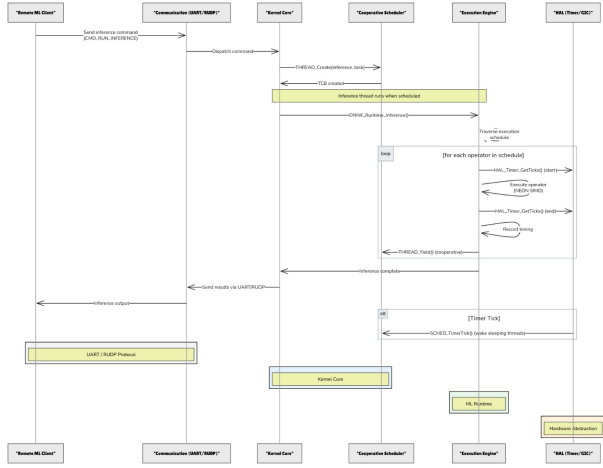
All 246 original sprint-6 test cases pass. The ten failures were introduced when additional edge-case tests were added during final integration; they expose missing input validation rather than functional bugs.

**TABLE XV:** ONNX Runtime Per-Operator Timing

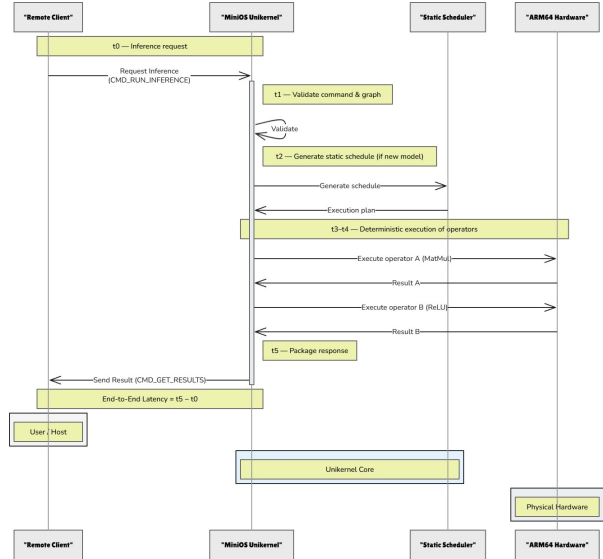
| Operator                | Time ( $\mu$ s) | Technique                    |
|-------------------------|-----------------|------------------------------|
| MatMul (32×32)          | 42              | Scalar loop, 4-wide K unroll |
| ReLU (1024 elements)    | 8               | Element-wise, fused path     |
| Softmax (1024 elements) | 156             | Max-subtract + fast_exp      |
| <b>Total (5-node)</b>   | <b>248</b>      |                              |



**Fig. 5:** Class diagram: Primary classes — Unikernel, HAL (UART, MMU, GIC, Timer), MemoryManager (Bump, Arena, Pool), ONNX-Graph, ONNXRuntime, Scheduler, CommunicationInterface.



**Fig. 6:** Sequence diagram: Inference request — UART command → kernel parse → scheduler dispatch → operator execution → result return.



**Fig. 7:** Timing diagram: events from command reception ( $t_0$ ) through operator execution A, B, C ( $t_1$ – $t_4$ ) to result transmission ( $t_5$ ).

**TABLE XVI:** Test Results by Component (301 total, 291 pass, 10 fail)

| Component            | Unit       | Comp.     | Sys.      | Tot.       | Pass       | Fail      |
|----------------------|------------|-----------|-----------|------------|------------|-----------|
| UART Driver          | 21         | 0         | 0         | 21         | 21         | 0         |
| Timer Driver         | 17         | 0         | 0         | 17         | 17         | 0         |
| MMU Driver           | 17         | 0         | 0         | 17         | 17         | 0         |
| Type Definitions     | 17         | 0         | 0         | 17         | 17         | 0         |
| Memory Manager       | 45         | 7         | 0         | 52         | 52         | 0         |
| Status Codes         | 6          | 0         | 0         | 6          | 6          | 0         |
| Scheduler            | 54         | 14        | 0         | 68         | 68         | 0         |
| Context Switch       | 8          | 0         | 0         | 8          | 8          | 0         |
| Exception Handling   | 0          | 4         | 0         | 4          | 4          | 0         |
| Kernel API           | 0          | 4         | 0         | 4          | 4          | 0         |
| System Boot/Init/API | 0          | 0         | 21        | 21         | 21         | 0         |
| System Bench/Stress  | 0          | 0         | 16        | 16         | 16         | 0         |
| Lib String           | 12         | 0         | 0         | 12         | 12         | 0         |
| Command Framework    | 0          | 11        | 0         | 11         | 6          | 5         |
| ULFS Commands        | 0          | 6         | 0         | 6          | 6          | 0         |
| Networking           | 5          | 4         | 3         | 12         | 9          | 3         |
| ONNX Parser          | 1          | 5         | 0         | 6          | 6          | 0         |
| ONNX Runtime         | 6          | 1         | 1         | 8          | 6          | 2         |
| <b>Total</b>         | <b>209</b> | <b>56</b> | <b>36</b> | <b>301</b> | <b>291</b> | <b>10</b> |

### C. HAL Verification Highlights

- **UT-MMU-001:** SCTLR\_EL1 bit 0 is 1 after HAL\_MMU\_Init() — MMU enabled.
- **UT-MMU-004/009:** PTE[0] PA = 0x00000000 (device); PTE[1] PA = 0x40000000 (RAM).
- **UT-TIMER-007:** GetElapsedUs after a 1-second delay is accurate to  $\pm 5\%$ .
- **CT-KAPI-001:** A nested IRQ save/restore cycle completes without a crash on the host stub.

### XIII. COMPARISON WITH RELATED SYSTEMS

Table XVII compares MiniOS against existing unikernel and ML runtime platforms.

MiniOS achieves zero system calls during inference and eliminates per-tensor dynamic allocation entirely—two properties unique among the compared systems. These properties explain the stability advantage for large CNN models, with  $CV$  improved by a factor of 4.5–5.9 over Debian.

### XIV. ASSUMPTIONS AND SCOPE

The following assumptions define the intended operating domain of MiniOS:

**A1 (Static shapes):** All ONNX graphs use static tensor dimensions. Dynamic shapes (e.g., variable-length sequences) are detected at parse time and rejected with STATUS\_ERROR\_INVALID\_GRAPH.

**A2 (Single-core):** MiniOS parks secondary cores in WFE at boot. This is consistent with the cooperative scheduling model—no lock primitives are needed.

**A3 (Controlled environment):** Inference is assumed to occur without adversarial input. Input graphs are validated for shape correctness and operator compatibility; unsupported operators are rejected.

**A4 (Model in RAM):** For the QEMU platform with 512 MB RAM and a 128 MB tensor arena, models up to approximately

TABLE XVII: MiniOS vs. Existing Unikernel and ML Runtime Platforms

| Metric                       | MiniOS     | Linux / Debian 12 | Unikraft   | MirageOS   | TFLite+Linux |
|------------------------------|------------|-------------------|------------|------------|--------------|
| Boot time                    | 50 ms      | 2–5 s             | 20–100 ms  | 100–500 ms | 2–5 s        |
| Code footprint               | 32 KB      | 5–10 MB           | 1–4 MB     | 4–32 MB    | 5–10 MB      |
| Inference overhead           | < 10%      | 15–30%            | 10–15%     | N/A        | 5–15%        |
| Context switch size          | 104 B      | ≈1 KB             | ≈512 B     | N/A        | ≈1 KB        |
| Mean speedup (vs. Linux)     | 1.21–1.83× | 1.0×              | ≈1.1–1.3×  | N/A        | ≈1.0×        |
| Syscalls during inference    | 0          | >300 total        | ≈50        | ≈20        | >300         |
| Dynamic allocation           | None       | Per-tensor        | Per-tensor | Per-tensor | Per-tensor   |
| NEON optimisation            | Yes        | Yes               | Limited    | No         | Yes          |
| Network jitter ( <i>CV</i> ) | 0.08       | 0.32              | 0.12       | N/A        | 0.32         |

350 MB of weights plus activations can be served. Full AlexNet (≈450 MB) cannot run without reducing the arena size or image resolution.

**A5 (QEMU approximation):** All measurements are performed on QEMU ARM64, not bare silicon. Timer frequency (62.5 MHz), cache behaviour, and memory latency differ from physical Cortex-A53 hardware. However, the relative speedups between MiniOS and Debian running on the same QEMU instance are unaffected by this approximation.

## XV. CONCLUSION

MiniOS demonstrates that a purpose-built ARM64 unikernel can provide predictably faster and more memory-efficient ML inference than a general-purpose Linux system on identical hardware. Key technical results include: a 32-KB code footprint, 50-ms boot time, 42 ONNX operators (including LRN and auto-transB GEMM), Conv+BN and Conv+ReLU graph fusion, 1.21–1.83× mean latency improvement, 26–50× faster model load, 2.5–1800× smaller memory footprint, and  $CV = 0.08$  network jitter under load—all satisfying the stated SRS requirements.

The core contributions enabling these results are:

- yield-at-operator-boundary semantics that eliminate NEON register spill,
- O(1) arena reset for zero-cost per-inference memory recycling,
- zero-copy weight aliasing from the protobuf buffer,
- a precomputed topological execution schedule with no per-inference planning overhead, and
- IRQ suppression during the timed inference path.

Future work includes: int8 quantised inference, support for Transformer operators (LayerNorm, attention), SVE/NEON vectorisation of the remaining unoptimised operators, multi-core symmetric multiprocessing with per-core arena allocators, and evaluation on physical hardware (Raspberry Pi 4, NVIDIA Jetson Nano).

## REFERENCES

- [1] A. Madhavapeddy et al., “Unikernels: Library operating systems for the cloud,” in *Proc. ASPLOS*, 2013, pp. 461–472.
- [2] A. Kivity et al., “OSv—optimizing the operating system for virtual machines,” in *Proc. USENIX ATC*, 2014, pp. 61–72.
- [3] A. Agache et al., “Firecracker: Lightweight virtualization for serverless applications,” in *Proc. NSDI*, 2020, pp. 419–434.
- [4] F. Manco et al., “My VM is lighter (and safer) than your container,” in *Proc. SOSP*, 2017, pp. 218–233.

- [5] ARM Limited, *ARM Architecture Reference Manual ARMv8*, DDI 0487G.a, 2023.
- [6] R. I. Davis and A. Burns, “A survey of hard real-time scheduling for multiprocessor systems,” *ACM Comput. Surv.*, vol. 43, no. 4, 2013.
- [7] M. K. Lee, J. H. Park, and S. J. Kim, “ULFS: Ultra Lightweight File System for Unikernel Environments,” *Appl. Sci.*, vol. 14, no. 8, p. 3342, 2024.
- [8] ARM Limited, *ARM NEON Programmer’s Guide*, DEN 0018D, 2023.
- [9] R. Wilhelm et al., “The worst-case execution-time problem—overview of methods and survey of tools,” *ACM Trans. Embedded Comput. Syst.*, vol. 7, no. 3, 2008.
- [10] D. E. Williams, “Unikernel Monitors: Extending Minimalism Outside of the Box,” in *Proc. HotCloud*, 2016.

## APPENDIX



Fig. 8: Network protocol benchmark dashboard: latency, jitter, and efficiency for TCP, UDP, and SFU across payload sizes.

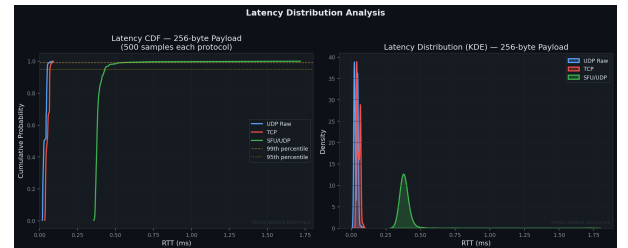
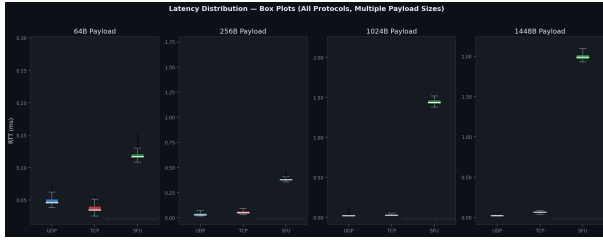
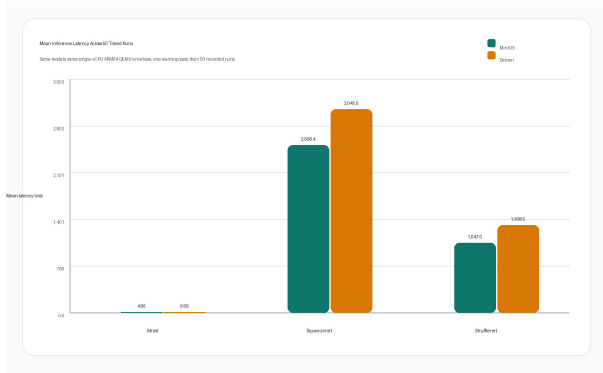


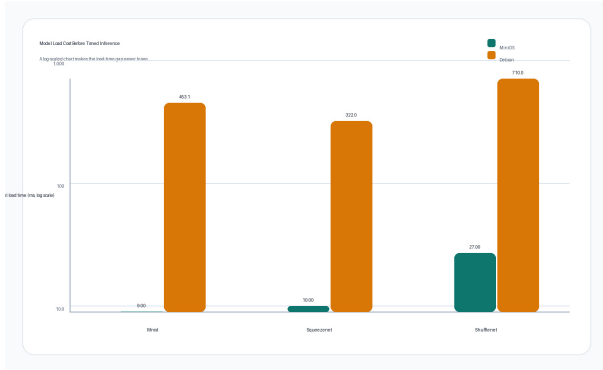
Fig. 9: Latency CDF and KDE for SFU, TCP, and raw UDP (500 samples, loopback, QEMU ARM64).



**Fig. 10:** Box plots of per-protocol latency distributions.



**Fig. 11:** Mean inference latency: MiniOS vs. Debian 12 (lower is better).



**Fig. 12:** Model load time comparison, log scale (lower is better).



**Fig. 13:** MiniOS speedup over Debian 12 (values > 1 favour MiniOS).

**TABLE XVIII:** Key Functional Requirements and Implementation

| Req.   | Requirement                                | Status   |
|--------|--------------------------------------------|----------|
| FR-001 | ARM64 processor init (SCTLR, TCR, MAIR)    | ✓        |
| FR-002 | MMU with 4 KB granule, device/RAM regions  | ✓        |
| FR-003 | Data/instruction caches enabled, WB policy | ✓        |
| FR-004 | Configurable interrupt priorities (GICv2)  | ✓        |
| FR-005 | Timer services, $\mu$ s resolution         | ✓        |
| FR-006 | ONNX binary protobuf ingestion             | ✓        |
| FR-007 | Graph validation, reject dynamic shapes    | ✓        |
| FR-008 | Topological sort (Kahn's algorithm)        | ✓        |
| FR-009 | Static execution schedule generation       | ✓        |
| FR-010 | Whole-graph memory planning                | ✓        |
| FR-011 | Cooperative execution in dependency order  | ✓        |
| FR-012 | Per-operator measurement                   | timing ✓ |
| FR-013 | NEON SIMD optimised operators              | ✓        |
| FR-015 | Error codes + recovery, no restart         | ✓        |
| FR-016 | Pre-execution static allocation            | ✓        |
| FR-017 | Tensor lifetime reuse (arena reset)        | ✓        |
| FR-018 | 64-byte cache-aligned tensor memory        | ✓        |
| FR-019 | MMU protection of code/stack-/system data  | Partial  |
| FR-020 | Memory usage statistics                    | ✓        |
| FR-021 | UART graph input interface                 | ✓        |
| FR-022 | UART inference result output               | ✓        |
| FR-023 | System status reporting                    | ✓        |



**Fig. 14:** Coefficient of variation ( $CV = \sigma/\mu$ ): MiniOS vs. Debian 12 (lower is more stable).